Redes de Computadores

1st Semestre, 2026/2027
LEIC Alameda

Programming using the *Sockets* interface

"NetBoX"
Phase I

Date: 10 September 2026

1. Introduction

The goal of this project is to implement a distributed file storage and sharing platform, called **NetBoX**, using the sockets interface. The platform allows users to publish information about files they are willing to share, search for files published by other users, inspect different published versions, and download files directly from those users. **NetBoX** combines centralized coordination with decentralized peer-to-peer file transfer.

The development of the project requires implementing a **User Application**. A **Directory Server (*DS*)** will be provided by the course instructors and will run on a machine with a known IPv4 address and port. The *DS* stores registered users, active sessions, and resource metadata, but never stores the actual file contents. Files remain stored locally at the users' machines and are transferred directly between User applications.

Students are responsible for implementing the **User Application** only. Multiple User application instances are intended to operate simultaneously on different machines connected to the Internet.

Each User application performs multiple roles:

- UDP client of the *DS* for user management, publication, removal, listing, and peer-location requests;
- TCP client of the *DS* for receiving information about existing resources;
- TCP peer client for downloading a selected file from another User application.
- TCP peer server for handling incoming file-transfer requests;

Project evaluation is **continuous throughout the semester** and **includes periodic checkpoints**, **intermediate assessments**, and **a final project demonstration and discussion**. Students are therefore expected to make regular progress during the semester.

Attendance of lab classes is mandatory.

2. Project Description

The User application interface allows controlling the actions to take through keyboard commands, including login, logout, publishing files, removing published files, listing available resources, searching for resources, and downloading files from peers.

- Login (*login*). Each user is identified by a *user ID*, `UID`, a 6-digit IST student number, and a *password* composed of 8 alphanumeric (only letters and digits) characters. When the *DS* receives a login request it will inform of a successful or incorrect login attempt or, in case the `UID` was not yet present at the *DS*, it registers a new user.
- Publish a resource (*publish*). The *User* application informs the *DS* that a file available in the local machine should be advertised to other users. The *DS* stores information associating the published file with the corresponding user. The command also sends information about the shared file, including size of file and its type (label).
- Remove a resource (*remove*). The *User* application informs the *DS* that a previously published resource should no longer be advertised to other users.
- List resources (*list*). The *User* application asks the *DS* for a list of the resources currently known in the network. The *DS* replies with the requested list or indicates that no resources are currently available. The list only contains file names.
- List different versions (*versions*). The *User* application asks the *DS* for the list of existing versions of resource (file). The *DS* replies with the requested list or indicates that no resources are currently available. The list includes all users who announced this resource, including size, label, time, and availability.
- Search for a resource (*where*). The *User* application asks the *DS* for Location of a specific resource. The *DS* replies with the network location of the user that has previously published the requested resource and is currently available.
- Download a resource (*get*). The *User* application contacts another *User* application directly and requests the transfer of the selected resource. If the resource exists and is available, the file is transferred directly between the two peers.
- Unregister (*unregister*). The *User* application asks the *DS* to unregister this *user* (the currently logged-in user). Resources previously published by that user remain stored in the *DS* database but are no longer associated with an active user session.
- Logout (*logout*). The user can ask the *DS* server to terminate the current user's session.
- Exit (*exit*). The user can ask to exit the *User* application. If a user is still logged in, then the *User* application informs the user that the logout command should be executed first.

The implementation uses application-layer protocols operating over the transport-layer services made available by the sockets interface. Communication with the Directory Server is supported using UDP and TCP, while file transfer between users is supported using TCP.

The objective of the project is not only to implement a working distributed file sharing application, but also to understand protocol design, transport-layer trade-offs, peer-to-peer communication, resource discovery, and event-driven socket programming.

2.1. Progressive development

The project is developed progressively throughout the semester.
Phase I: implement the UDP client operations needed for login, logout, and unregister. Students are initially required to implement simple communication with the Directory Server (using UDP). Students develop a working UDP client that communicates with the UDP server provided by the course instructors.

First phase *should be concluded in* ***week 2.***

3. Project Specification – *Phase I*

3.1. User Application (*User*)

The program implementing the *users* of the NetBox (*User*) is invoked using:

`./user -m peerport [-n DSIP] [-p DSport]`,

where:

`peerport` is the mandatory TCP port where the User application accepts incoming file-transfer requests from other users. This is a mandatory argument.

`DSIP` is the IP address of the machine where the Directory Server (*DS*) runs. This is an optional argument. If this argument is omitted, the application should assume the DSIP value provided by the professors.

`DSport` is the well-known UDP port where the *DS* accepts requests. This is an optional argument. If omitted, it assumes the UDP port value provided by the professors.

Once the *User* application is running, the user can login, logout, unregister.
During the first login of a user, its identifier, `UID`, is used to register this user in the *DS* database. The *user* also has the possibility to logout, unregister or exit the *User* application.

The commands supported by the *User* interface (using the keyboard for input) are:

- ***`login`*** `UID password` – following this command, the *User* application sends a message to the *DS*, using the UDP protocol. The *DS* confirms the ID, `UID`, and `password` of this user, or registers it if this `UID` is not present in the *DS* database. The result of the request should be displayed: successful login, incorrect login attempt, or new user registered.
- ***`unregister`*** – following this command, the *User* application sends a message to the *DS*, using the UDP protocol, asking to unregister the currently logged-in user. A logout operation is also performed.
  The result of the request should be displayed: successful unregister, unknown user, or incorrect unregister attempt.
- ***`logout`*** – following this command, the *User* application sends a message to the *DS*, using the UDP protocol, asking to logout the currently logged-in user, with ID `UID`. Published resources remain stored in the *DS* database, but the user is no longer considered active and will not be returned as an available peer until a new successful login is performed.
  The result of the request should be displayed: successful logout, unknown *user*, or *user* not logged in.
- ***`exit`*** – this is a request to terminate the *User* application. If a user is still logged in the *User* application should inform the user to first execute the *logout* command. Otherwise, the application can be terminated. *This is a local command, not involving communication with the DS.*

Only one messaging command can be issued at a given time.
The result of each interaction with the *DS* should be displayed to the user.

3.2. Directory Server (*DS*)

The program implementing the ***Directory Server* (*DS*)** is provided by the course instructors. Students **are not required** to implement a version of this server.

The *DS* runs a UDP server and a TCP server responsible for managing users and maintaining information about the resources available in the network. The *DS* stores information about registered users, currently logged-in users, and the resources published by those users.

For each logged-in user, the *DS* maintains information about the user's network location, including the IP address and the TCP port where the User application accepts incoming file-transfer requests.

Currently, the ***Directory Server* (*DS*)** is running at the server tejo, in the laboratory LT5.

Information of ***DS***:

Parameter	Value	Comments
IP of *DS*	193.136.138.142 (Public IP of tejo)	When accessing from anywhere other than LT5
IP of *DS*	192.168.1.1	When accessing the server from the laboratory LT5
Port (UDP) of *DS*	59000	To use in UDP commands

4. Communication Protocols Specification – *Phase I*

The communication protocols used to implement the distributed file storage are described in this section. For the communication protocols `UID` is always sent using 6 digits (*only digits*).

4.1. *User–DS* Protocol (in UDP)

In this phase, the interaction between the user application (*User*) and the Directory Server (*DS*) is supported by the UDP protocol. The related request and reply protocol messages are described in this section.

For the communication protocols, UID is always sent using 6 digits and password is always composed of 8 alphanumeric characters.

a) `LIN UID password peerTCPport`
Following the ***login*** command, the *User* application informs the *DS* that user `UID`, with password `password`, wants to login, to use the application. The field `peerTCPport` indicates the TCP port where this *User* application accepts incoming file-transfer requests from other users.

b) `RLI status`
In reply to a `LIN` request the *DS* checks if a user with ID `UID` is already registered. If so, the password `password` is checked, and for a correct match the *DS* takes note that the *user* is now logged in and replies with `status` = `OK`. For an incorrect password, the reply `status` is `NOK`. If the *user* was not yet registered, the *DS* registers and logs in this *user*, storing the corresponding `UID, password`, network location and peer TCP port. The reply `status` is then `REG`.

c) `LOU UID password`
Following the ***logout*** command the *User* application informs the *DS* that the currently logged-in user, with ID `UID`, wants to logout.

d) `RLO status`
In reply to a `LOU` request, the *DS* checks if a user with ID `UID` is logged in. If so, the *user* is logged out and the reply `status` is `OK`. If the user was not logged in, the reply `status` is `NLG`. If the *user* was not registered, the reply `status` is `UNR`. If the password is incorrect, the reply `status` is `WRP`.

e) `UNR UID password`
Following the ***unregister*** command, the *User* informs the *DS* that the currently logged-in user, with ID `UID`, wants to unregister.

f) `RUR status`
In reply to a `UNR` request, the *DS* checks if a user with ID `UID` is registered. If so, the user can be unregistered and reply `status` is `OK`. If the user was not logged in, the reply `status` is `NOK`. If the user was not registered, the reply `status` is `UNR`. If the password is incorrect, the reply `status` is `WRP`.
When a user is successfully unregistered, a logout operation is also performed.

The *User* application should always display information about the received replies in a human friendly format.
In the above messages the separation between any two items consists of a single space.
`* [field]` means that `field` is optional.

In the above messages the separation between any two items consists of a single space.
Each request or reply message ends with the character "`\n`".

For replies including the `status` field it takes the value `ERR` when the syntax of the request message was incorrect or when the parameter values take invalid values.

If an unexpected protocol message is received, the reply is `ERR`.

4.2. *Peer-to-Peer* Messaging Protocol (in TCP)

The interaction between two user applications (*User*) for the transfer of files is supported by the TCP protocol.
The related request and reply protocol messages using TCP will be provided in later phases.

4.3. *Validation and Limits*

Field/item	Requirement
UID	Exactly 6 decimal digits (only digits)
Password	Exactly 8 ASCII alphanumeric characters
Port	Decimal integer from 1 to 65535

5. Development

5.1. Development and test environment

Make sure your code compiles and executes correctly in the development environment available in the lab *LT5*.

If the file does not compile correctly in the development environment of lab LT5, marks will be deducted from the final grade of the laboratory part.

5.2. Programming

The operation of your program, developed in *C* or *C++*, may need to use the following set of system calls:

- Reading user information into the application: `fgets()`;
- Manipulation of strings: `sscanf()`, `sprintf()`;
- UDP client management: `socket()`, `close()`;
- UDP server management: `socket()`, `bind()`, `close()`;
- UDP communication: `sendto()`, `recvfrom()`;
- TCP client management: `socket()`, `connect()`, `close()`;
- TCP server management: `socket()`, `bind()`, `listen()`, `accept()`, `close()`;
- TCP communication: `write()`, `read()`;
- Multiple inputs multiplexing: `select()`.

5.3. Implementation notes

Developed code should be adequately structured and commented.

The `read()` and `write()` system calls may read and write, respectively, a smaller number of bytes than solicited – you need to ensure that your implementation still works correctly.

The client and the server processes should not terminate abruptly in failure situations, such as these:

- wrong protocol messages received by the server: an error message should be returned, as specified by the protocol;
- wrong protocol messages received by the client: the interaction with the server is not continued and the user is informed;
- error conditions from system calls: the programs should not terminate abruptly, avoiding cases of "segmentation fault" or "core dump".

6. Bibliography

- W. Richard Stevens, Unix Network Programming: Networking APIs: Sockets and XTI (Volume 1), 2nd edition, Prentice-Hall PTR, 1998, ISBN 0-13-490012-X, chap. 5.
- D. E. Comer, Computer Networks and Internets, 2nd edition, Prentice Hall, Inc, 1999, ISBN 0-13-084222-2, chap. 24.
- Michael J. Donahoo, Kenneth L. Calvert, TCP/IP Sockets in C: Practical Guide for Programmers, Morgan Kaufmann, ISBN 1558608265, 2000
- On-line manual, `man` command

7. Project Submission

7.1. *Code*

The project submission should include the source code of the programs implementing the *User* application, as well as the corresponding *Makefile* and the *auto-avaliação* excel file. The *makefile* should compile the code and place the executables in the current directory.

7.2. *Auxiliary Files*

Together with the project submission you should also include any auxiliary files needed for the project operation together with a *readme.txt* file.

7.3. *Submission*

The project submission is done by e-mail to the lab teacher, **no later than December 4, 2026, at 23:59 PM**.
You should create a single `zip` archive containing all the source code, makefile, all auxiliary files required for executing the project and the *auto-avaliação* excel file. The archive should be prepared to be opened to the current directory and compiled with the command `make`.
The name of the archive should follow the format: **`proj_groupnumber.zip`**

8. Open Issues

You are encouraged to think about how to extend this protocol in order to make it more generic.